Out-of-Thin-Air Execution is Vacuous

ISO/IEC JTC1 SC22 WG21 P0422R0 - 2016-07-27

Paul E. McKenney, paulmck@linux.vnet.ibm.com
Alan Jeffrey, ajeffrey@mozilla.com
Ali Sezgin, as2418@cam.ac.uk
Tony Tye, Tony.Tye@amd.com

Introduction

This paper is an updated version of N4375, revised based on discussions with Tony Tye. N4375 itself is based on N4323, revised based on discussions with Michael Wong and Maged Michael. N4323 was in turn an updated version of N4216, revised based on discussions at the 2014 UIUC meeting (with much good feedback especially from Hans Boehm and Victor Luchangco) and on email reflector discussion.

Out-of-thin-air (OOTA) values have proven to be a thorny issue for memory models, including the Java memory model (JMM) and the C11 and C++11 memory models. The current C and C++ draft standards simply advise implementers to avoid OOTA values, without precisely defining what OOTA values might be. A number of publications have looked at this, including that of Vafeiadis et al., Batty et al., Boehm and Demsky, Sevcik, Jeffrey, Sewell and Batty, and the JMM Causality Test Cases. These publications establish that OOTA is harmful, and look at a number of interesting consequences. Unfortunately, these publications focus only on a small (relatively) sensible-seeming subset of possible OOTA scenarios. This paper will explore some of the less sane scenarios, which will have the side-effect of demonstrating that out-of-thin-air execution is, in the C and C++ worlds, vacuous.

To that end, this paper will look at an interesting open problem, which is the fact that harmful OOTA programs can be very closely related to benign operation-reordering programs. This paper will discuss a general method, called *perturbation analysis* that may be used to distinguish harmful from benign.

NOTE: There are no known production-quality implementations that can induce OOTA. Therefore, implementations need not do anything to avoid OOTA, other than avoid dubious (at best) data-speculation optimizations. The OOTA shortcoming is therefore a shortcoming of theory rather than a change needed in practice.

Examples: Harmful OOTA vs. Benign Reordering

The canonical harmful-OOTA example is as follow, where `x` and `y` are both initially zero, and where all accesses to shared variables are `memory_order_relaxed` (though loads may instead be `memory_order_consume` loads):

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
y = r1;                 x = r2;
```

The current C and C++ standards do not rule out the outcome of `r1` and `r2` both equalling 42—or any other value that can be represented by `x` and `y`. This outcome would of course be quite surprising, and would have a number of fatal consequences.

In contrast, the following closely related program is an example of benign reordering:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
y = 42;                 x = r2;
```

Here, the outcome of `r1` and `r2` is perfectly legitimate, and in fact occurs on actual implementations.

However, the presence of the constant 42 in and of itself cannot distinguish between the benign and harmful cases, for example, this program is an example of harmful OOTA:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
if (r1 == 42)           x = r2;
    y = 42;
else
    y = r1;
```

This example can be extended to produce a variety of OOTA values by expanding Thread 1's "if" statement to provide additional values. A very large number and variety of examples can be generated, a few of which appear in the the JMM Causality Test Cases.

Properties of Harmful OOTA

In the canonical harmful OOTA example, the value of 42 comes from nowhere, and circulates between `x` and `y`. This situation suggests the following perverse modification, which assumes that `x` and `y` are unsigned and thus not subject to undefined behavior upon overflow:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
y = r1;                 x = r2 + 1;
```

This cannot result in `r1` and `r2` both having the value 42. To see this, note that the only way that `r1` can have a non-zero value is if it loads the value stored by Thread 2. Similarly, the only way that `r2` can have a non-zero value is if it loads from Thread 1's store. So suppose that `r2` has the value 42. This means that Thread 2 stores 43, which means that the value of `r1` will also be 43. But this means that Thread 1 will store 43 to `y`, which means that `r2` also cannot be 42, contradicting the initial assumption. This example demonstrates that OOTA execution is similar to the classic spreadsheet "solve" functionality: OOTA conceptually requires iterating until a fixed point is reached. This functionality has its place in spreadsheets, but has no place in the confines of the C and C++ memory models, most especially for non-converging test cases such as above. Hence, this example demonstrates that OOTA is not just confusing and harmful (for example, by inflicting undefined behavior on unsuspecting developers and code), but is also vacuous in the context of the C and C++ memory models.

Please note that this problem does not occur if either or both of the loads get the initial value of zero. The three convergent cases are as follows:

r1 Source	r2 Source	r1	r2	x	y
Initial Value	Initial Value	0	0	1	0
Initial Value	Thread 1 Store	0	0	1	0
Thread 2 Store	Initial Value	1	0	1	1

In contrast, the benign example does not diverge when modified:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
y = 42;                 x = r2 + 1;
```

This program produces well-defined results regardless of where the loaded values come from, as shown below:

r1 Source	r2 Source	r1	r2	x	y
Initial Value	Initial Value	0	0	1	42
Initial Value	Thread 1 Store	0	42	43	42
Thread 2 Store	Initial Value	1	0	1	42
Thread 2 Store	Thread 1 Store	43	42	1	42

The if-statement version of the harmful example also diverges in response to perturbation:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
if (r1 == 42)           x = r2 + 1;
    y = 42;
else
    y = r1;
```

If Thread 2 reads from Thread 1's store, it might see the store of the constant 42. In that case, it will store 43 to `x`. But if Thread 1 also reads Thread 2's store, it will load the value 43, and thus won't execute the store of 42, which means that Thread 2's load gives 43, not 42, contradicting the initial assumption.

General Two-Thread/Two-Variable OOTA

A general two-thread/two-variable form of OOTA is as follows:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
if (f(r1, &r2))         if (g(r3, &r4))
    y = r2;                 x = r4;
```

The previous section applied a perturbation function to Thread 2's store, resulting in the following:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
if (f(r1, &r2))         if (g(p(r3), &r4))
    y = r2;                 x = r4;
```

In the previous section, we chose `p()` to the increment function. Unfortunately, this choice does not always force an inconsistency, for example:

```
int f(unsigned a, unsigned *b)
{
    *b = a & ~0x1;
    return 1;
}

unsigned g(unsigned a, unsigned *b)
{
    *b = a;
    return 1;
}

unsigned p(unsigned a)
{
    return a + 1;
}

Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
if (f(r1, &r2))         if (g(p(r3), &r4))
    y = r2;                 x = r4;
```

If Thread 1 stores the value 42 to `y`, Thread 2 will increment it and thus store 43 to `x`. But Thread 1's call to `f()` will strip off the bottom bit, restoring both the value 42 and consistent execution. In this case, a perturbation function `p` that is an increment fails to force an inconsistency (although it does succeed in changing the overal behavior). The choice of the perturbation function `p()` depends on the algorithm, and is in the general case undecidable.

However, all is not lost. First, it can easily be seen that a function `p()` that increments by two rather than one suffices to produce the needed inconsistency. This is still a total function and results in the following, where the functions `f()`, `g()`, and `p()` have all been inlined for ease of exposition:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
y = r1 & ~0x1;          x = r2 + 2;
```

Here, if Thread 2's load returns 42, it will store the value 44. Thread 1's load will thus return 44, which is unaffected by the bitwise AND, so that Thread 1 stores 44. This contradicts Thread 2's initial load of 42, thus providing the needed inconsistency.

Although the choice of `p()` is in theory undecidable, the examples in this paper can be solved for a suitable `p()` using (at most) simple algebra. We further conjecture that a randomly chosen function would have a high probability of forcing an inconsistency. In fact, it is possible for the identity function to result in an inconsistency, for example, in the following case:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
y = r1;                 x = ~r2;
```

Because the example is itself inconsistent, the choice of the identity function for `p()` suffices to preserve this inconsistency.

We further conjecture that the choice of `p()` is not only decidable but trivial in the case where all variables are boolean. In this case, `p()` can be simple boolean NOT, as in the C and C++ prefix `!` operator. In fact, the only reasonable choices for `p()` are NOT on the one hand and the identity function on the other. It might be necessary to apply `p()` to the Thread 1's load instead of that of Thread 2. Of course, just as with integers, it is necessary to check the original example for inconsistencies before applying a perturbation function.

We will see that the choice of perturbation function `p()` is constrained as follows:

1. `p()` must be total over the set of possible argument values.
2. `p()` must not violate constraints deduced from global analysis.

JMM Causality Test Cases

This section applies perturbation to each of the JMM causality test cases, comparing the results to the decisions.

Causality Test Case 1

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
if (r1 >= 0)            x = r2;
    y = 1;

Behavior in question: r1 == r2 == 1
Decision: Allowed.
```

The decision is based on the assumption that the compiler determines that the variables are all non-negative. We can define `p()` to be the increment function, and see that although this choice of perturbation function does change the behavior, it does not introduce an inconsistency.

On the other hand, if we violate the non-negativity assumption by choosing `p()` to be the function that decrements by two, we have `r2 == 1 && x == -2 && r1 == -2 && y == 0`, which is an inconsistent execution.

This example therefore illustrates another constraint on the perturbation function, namely that it not violate constraints deduced from global analysis.

Causality Test Case 2

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
r2 = x;                 x = r3;
if (r1 == r2)
    y = 1;

Behavior in question: r1 == r2 == r3 == 1
Decision: Allowed.
```

Assume an arbitrary perturbation function `p()`, and that Thread 1's loads happen after Thread 2's store. Then `r1` will always be equal to `r2`, so that Thread 1 will always store to `y`. Therefore, we have a consistent execution regardless of the perturbation function.

Causality Test Case 3

```
Thread 1                Thread 2                Thread 3
--------                --------                --------
r1 = x;                 r3 = y;                 x = 2
r2 = x;                 x = r3;
if (r1 == r2)
    y = 1;

Behavior in question: r1 == r2 == r3 == 1
Decision: Allowed.
```

Assume an arbitrary perturbation function `p()`, and that Thread 1's loads happen after both Thread 2's and Thread 3's stores. Then `r1` will always be equal to `r2`, so that Thread 1 will always store to `y`. Therefore, we again have a consistent execution regardless of the perturbation function.

Causality Test Case 4

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
y = r1;                 x = r2;

Behavior in question: r1 == r2 == 1
Decision: Disallowed.
```

This test case was analyzed earlier, and that analysis agrees with the decision of “disallowed.” Interestingly enough, a compiler examining this test case could deduce that only the value 0 is assigned to `x` and `y` (at initialization time). The JMM applied this sort of compiler-based variable-value deduction to other test cases, so it is curious that they chose not to apply it to this case. (Or, alternatively, given that they did not apply it to this case, it is curious that they felt comfortable applying it to other cases.) Of course, in general, the range of values of variables is also undecidable.

Causality Test Case 5

```
Thread 1                Thread 2                Thread 3                Thread 4
--------                --------                --------                --------
r1 = x;                 r2 = y;                 z = 1;                  r3 = z;
y = r1;                 x = r2;                                         x = r3;

Behavior in question: r1 == r2 == 1, r3 == 0
Decision: Disallowed.
```

Because `r3` is zero, we know that Thread 4 stored zero to `x`. Therefore, the only way for `r1` and `r2` to equal 1 is for an OOTA cycle involving only Threads 1 and 2. However, this part of the test case is the same as test case 4, and perturbation analysis gives the same outcome of “disallowed.”

Causality Test Case 6

```
Thread 1                Thread 2
--------                --------
r1 = A;                 r2 = B;
if (r1 == 1)            if (r2 == 1)
    B = 1;                  A = 1;
                        if (r2 == 0)
                            A = 1;

Behavior in question: r1 == r2 == 1
Decision: Allowed.
```

`B` is always either zero or one, so Thread 2 will load either zero or one into `r2`. This means that one or the other of the two `if` statements will always be taken, so Thread 2 will always store the value 1 to `A`. This means that a sufficiently aggressive compiler could eliminate Thread 2's `if` statements and simply unconditionally assign to `A`. Because all memory references are relaxed, the order of Thread 2's load and store can be reversed, after which the result is allowed even in an SC execution.

Perturbation can chance the values of `r1` and `r2`, but cannot introduce inconsistencies. If the compiler cannot determine that `B` is always either zero or one, perturbation still cannot introduce inconsistencies. Either way, perturbation analysis agrees with the decision of “allowed.”

Causality Test Case 7

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
r2 = x;                 z = r3;
y = r2;                 x = 1;

Behavior in question: r1 == r2 == r3 == 1
Decision: Allowed.
```

Simple reordering can produce the behavior, and adding perturbation can change the behavior, but cannot result in inconsistencies. For example, applying an arbitrary perturbation function `p()` to the value stored to `y` results in the following:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
r2 = x;                 z = r3;
y = p(r2);              x = 1;
```

In this case, we can see `r1 == r2 == 1 && r3 == p(1)`. So because perturbation does not introduce inconsistencies (instead merely changing the behavior), perturbation analysis agrees with the decision of “allowed.”

Causality Test Case 8

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
                        x = r3;
r2 = 1 + r1 * r1 - r1;
```

```
y = r2;

Behavior in question: r1 == r2 == 1
Decision: Allowed.
```

One wonders why `r3 = 1` was not included in the behavior.

The analysis in the JMM test cases assumes that inter-thread analysis determines that `x` and `y` is always either zero or one, so that the compiler converts the code to the following:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
r2 = 1;                 x = r3;
y = r2;
```

In this case, perturbation results in the following:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
r2 = 1;                 x = p(r3);
y = r2;
```

Note that applying the perturbation function to Thread 1 has no effect: The `r1` variable is dead code. Applying the perturbation to Thread 2 causes the value 2 to be stored to `x`, which again has no effect in Thread 1 other than changing the value of `r1`. Therefore, perturbations do not result in inconsistency, which agrees with the decision of “allowed.”

This analysis applies given the range determination for `x` and `y` even without the optimization. In this case, the only reasonable perturbation function is the `!` operator, resulting in the following:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
                        x = !r3;
r2 = 1 + r1 * r1 - r1;
y = r2;
```

In this case, Thread 1's load from `x` returns either zero or one, but it will always store the value of one to `y`. This means that Thread 2's load from `y` will always return the value 1, so that there is no inconsistency. This again means that this test case is an example of benign reordering rather than harmful OOTA, again agreeing with the decision of “allowed.”

Causality Test Case 9

```
Thread 1                Thread 2                Thread 3
--------                --------                --------
r1 = x;                 r3 = y;                 x = 2
                        x = r3;
r2 = 1 + r1 * r1 - r1;
y = r2;

Behavior in question: r1 == r2 == 1
Decision: Allowed.
```

If the compiler can determine that Thread 3 executes only after both Threads 1 and 2, then analysis proceeds as with test case 8 above. On the other hand, if Thread 3 can execute before Threads 1 and 2, then the compiler cannot limit the values of `x` and `y` to zero and one, and so the perturbation might proceed as follows:

```
Thread 1                Thread 2                Thread 3
--------                --------                --------
r1 = x;                 r3 = y;                 x = 2
                        x = r3 + 1;
r2 = 1 + r1 * r1 - r1;
y = r2;
```

If each of the Thread 1's and Thread 2's loads returns the value stored by the other thread, inconsistency results. For example, if we assume Thread 1 stores the value 1, then Thread 2 will store the value 2. But that would mean that Thread 1 would calculate and store the value 4, which is inconsistent with the assumption that Thread 2 loaded the value 1. Therefore, if the compiler is unable to determine that the values of `x` and `y` are limited to zero and one, then a load-store cycle is illegal.

This situation might seem a bit disturbing, but it in fact will help lead to key insight, namely that optimizations that replace computations with the equivalent constants are legal and cannot result in OOTA values.

Causality Test Case 9a

This variation has the initial value of `x` be two rather than zero:

```
Thread 1                Thread 2                Thread 3
--------                --------                --------
r1 = x;                 r3 = y;                 x = 0
                        x = r3;
r2 = 1 + r1 * r1 - r1;
y = r2;

Behavior in question: r1 == r2 == 1
Decision: Allowed.
```

This plays out very similarly to test cases 8 and 9: If the compiler can assign a constant to `r2`, then Threads 1 and 2 can legitimately load from each other's stores, otherwise they cannot.

Causality Test Case 10

```
Thread 1                Thread 2                Thread 3                Thread 4
--------                --------                --------                --------
r1 = x;                 r2 = y;                 z = 1;                  r3 = z;
if (r1 == 1)            if (r2 == 1)                                    if (r3 == 1)
    y = 1;                  x = 1;                                          x = 1;

Behavior in question: r1 == r2 == 1, r3 == 0
Decision: Disallowed.
```

Given that `r3` is equal to zero, we know that Thread 4's load could not have read from Thread 2's store (possibly due to Thread 2's store not having executed in the first place). We also know that Thread 4 did not store to `x`. This test case therefore can be analyzed by looking only at Threads 1 and 2. Perturbation then proceeds as follows:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y - 1;
if (r1 == 1)            if (r2 == 1)
    y = 1;                  x = 1;
```

Suppose that Thread 1's load returns the value that Thread 2 stored. Then Thread 1's `if` statement will execute the store in its `then` clause. If Thread 2's load in turn returns the value that Thread 1 stored, `r2` will be zero, which will mean that Thread 2's `if` statement will *not* execute the store in its `then` clause. But that means that Thread 1's load cannot possibly return the value that Thread 2 stored because nothing was stored. This inconsistency means that this test case is an example of harmful OOTA, which agrees with the JMM decision of “disallowed”.

Causality Test Case 11

```
Thread 1                Thread 2
--------                --------
r1 = z;                 r4 = w;
w = r1;                 r3 = y;
r2 = x;                 z = r3;
y = r2;                 x = 1;

Behavior in question: r1 == r2 == r3 == r4 == 1
Decision: Allowed.
```

We again assume that each load returns the value of the corresponding store from the other thread. This results in an update order of `x`, `y`, `z`, `w`. Because this is acyclic, perturbation cannot introduce an inconsistency, so this is an example of simple reordering, and not OOTA at all. Thus, perturbation analysis agrees with the JMM decision of “allowed.”

Causality Test Case 12

This test case has initial values of zero for `x` and `y`, 1 for `a[0]`, and 2 for `a[1]`.

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r3 = y;
a[r1] = 0;              x = r3;
r2 = a[0];
y = r2;

Behavior in question: r1 == r2 == r3 == 1
Decision: Disallowed.
```

Suppose as usual that each thread's load returns the value from the corresponding store by the other thread. We can perturb as follows:

```
Thread 1              Thread 2
--------              --------
r1 = x;               r3 = y;
a[r1] = 0;            x = r3 + 1;
r2 = a[0];
y = r2;
```

Given this perturbation, if Thread 2 loads the value 1 from `y`, then it will store the value 2 to `x`. Thread 1 will then load 2, and run off the end of array `a`, resulting in undefined behavior (or, if the array has three elements, uninitialized values). This is clearly inconsistent, so this is an example of harmful OOTA, which agrees with the JMM decision of “disallowed.”

Causality Test Case 13

```
Thread 1              Thread 2
--------              --------
r1 = x;               r2 = y;
if (r1 == 1)          if (r2 == 1)
    y = 1;                x = 1;

Behavior in question: r1 == r2 == 1
Decision: Disallowed.
```

The following perturbation works in this case:

```
Thread 1              Thread 2
--------              --------
r1 = x;               r2 = y;
if (r1 == 1)          if (r2 + 1 == 1)
    y = 1;                x = 1;
```

As before, suppose that each threads' load returns the value from the other thread's corresponding store. Then `r2` will be one, so that `r2 + 1` will not be equal to one, in turn meaning that Thread 2's store will not be executed. In this case, `r1` must be zero, so that Thread 1's store also is not executed. This means that `r2` cannot possibly have the value one, resulting in an inconsistency. This agrees with the JMM decision of “disallowed.”

Causality Test Case 14

In this test case, accesses to `y` use `memory_order_seqcst`.

```
Thread 1              Thread 2
--------              --------
r1 = a;               do {
if (r1 == 0)              r2 = y;
    y = 1;                r3 = b;
else                  } while (r2 + r3 == 0);
    b = 1;            a = 1;

Behavior in question: r1 == r3 == 1 && r2 == 0
Decision: Disallowed.
```

If Thread 2 leaves its loop due to Thread 1's store to `y`, the resulting synchronized-with relationship will force the load from `a` to happen before the store, so that `r1 == 0`. We therefore consider executions where Thread 2 leaves its loop due to Thread 1's store to `b`.

In this case, we can use the following perturbation:

```
Thread 1              Thread 2
--------              --------
r1 = a;               do {
if (r1 - 1 == 0)          r2 = y;
    y = 1;                r3 = b;
else                  } while (r2 + r3 == 0);
    b = 1;            a = 1;
```

Suppose that Thread 1's load from `a` returns the value stored by Thread 2. Then Thread 1 will store to `y`, which, as noted above, ensures that either Thread 2 never exits its loop or that there is a synchronized-with relationship between the store to and the load from `y`. Either outcome makes it impossible for Thread 1 to load the value from Thread 2's store to `a`, resulting in an inconsistency. This agrees with the JMM's decision of “disallowed.”

Causality Test Case 15

In this test case, accesses to `x` and `y` use `memory_order_seqcst`.

```
Thread 1              Thread 2              Thread 2
--------              --------              --------
r0 = x;               do {                  x = 1;
if (r0 == 1)              r2 = y;
    r1 = a;               r3 = b;
else                  } while (r2 + r3 == 0);
    r1 = 0;           a = 1;
if (r1 == 0)
    y = 1;
else
    b = 1;

Behavior in question: r0 == r1 == r3 == 1 && r2 == 0
Decision: Disallowed.
```

Just as with test case 14, we must consider cases where Thread 2 leaves its loop due to Thread 1's store to `b`. And a similar perturbation suffices:

```
Thread 1              Thread 2              Thread 2
--------              --------              --------
r0 = x;               do {                  x = 1;
if (r0 == 1)              r2 = y;
    r1 = a - 1;           r3 = b;
else                  } while (r2 + r3 == 0);
    r1 = 0;           a = 1;
if (r1 == 0)
    y = 1;
else
    b = 1;
```

Suppose that Thread 1's load from `x` returns the value stored by Thread 3 and that Thread 1's load from `a` returns the value stored by Thread 2. But this means that Thread 1 will store to `y`, which forces Thread 2's store to `a` to happen after Thread 1's load from `a`, thus forcing an inconsistency. Hence perturbation analysis agrees with the JMM's decision of “disallowed.”

Causality Test Case 16

```
Thread 1              Thread 2
--------              --------
r1 = x;               r2 = x;
x = 1;                x = 2;

Behavior in question: r1 == 2 && r2 == 1
Decision: Allowed.
```

An arbitrary perturbation function applied to either load from `x` has no effect on subsequent execution (for some definition of “subsequent”). Therefore, perturbation analysis cannot induce an inconsistency, which agrees with the JMM decision of “allowed.”

Causality Test Case 17

```
Thread 1              Thread 2
--------              --------
r3 = x;               r2 = y;
if (r3 != 42)         x = r2;
    x = 42;
r1 = x;
y = r1;

Behavior in question: r1 == r2 == r3 == 42
Decision: Allowed.
```

At the point where Thread 1 loads `x` into `r1`, it has either just loaded the value 42 from `x` or just stored the value 42 to `x`. Therefore, the compiler could simply set `r1` to the constant 42. Once it has done that, because relaxed accesses do not provide any ordering guarantees, the assignment to `r1` (as well as the subsequent store to `y` may be reordered. Note that this transformation might be a bit controversial, because as soon as the assignment of 42 to `r1` is moved to precede the store to `x`, the rationale for replacing the load from `x` with 42 disappears. For the purpose of this analysis, we will assume that relaxed loads and stores permit even this somewhat extreme reordering.

Given that transformation, no perturbation can change the value that Thread 1 stores to `y`, which eliminates any possibility of inconsistency. Perturbation analysis thus agrees with the JMM decision of “allowed.”

Causality Test Case 18

```
Thread 1              Thread 2
--------              --------
r3 = x;               r2 = y;
if (r3 == 0)          x = r2;
    x = 42;
r1 = x;
y = r1;

Behavior in question: r1 == r2 == r3 == 42
Decision: Allowed.
```

Given a compiler that could figure out that the only possible values that could be loaded from `x` are 0 and 42, the perturbation analysis is restricted to perturbing within these two values, which gives the same result as test case 17.

Causality Test Case 19

```
Thread 1              Thread 2              Thread 3
--------              --------              --------
join thread 3         r2 = y;               r3 = x;
r1 = x;               x = r2;               if (r3 != 42)
y = r1;                                         x = 42;

Behavior in question: r1 == r2 == r3 == 42
Decision: Allowed.
```

If the compiler is allowed to optimize across the `join`, this is the same as test case 17.

Causality Test Case 20

```
Thread 1              Thread 2              Thread 3
--------              --------              --------
join thread 3         r2 = y;               r3 = x;
r1 = x;               x = r2;               if (r3 == 0)
y = r1;                                         x = 42;

Behavior in question: r1 == r2 == r3 == 42
Decision: Allowed.
```

If the compiler is allowed to optimize across the `join`, this is the same as test case 18.

Causality Test Case Discussion

In all cases, perturbation analysis gives the same decision as did the JMM's deliberations. We therefore hypothesize that the analysis distinguishes benign reordering from harmful OOTA.

It is important to note that the perturbation-analysis approach sidesteps the issue of which compiler optimizations may be used in a given situation: Optimizations are applied first, and only then is perturbation analysis undertaken. However, this sidestepping has the benefit that perturbation analysis applies equally well to C, C++, and Java, despite the very different restrictions on optimizations across these three languages.

It would be nice to have a succinct description of the set of test cases in which perturbation functions introduced inconsistencies. Ali Sezgin pointed out this set is described by `rf ∪ sdep`, where `rf` is the reads-from relationship and `sdep` is “semantic dependence”, roughly defined as those dependency relationships in which at least some changes in the value at the head of the dependency relationship propagate through, resulting in a change at the tail of that relationship.

Prohibiting executions that have cycles in `rf ∪ sdep` can therefore be expected to prohibit OOTA behaviors.

One beneficial consequence of this relationship to semantic dependency is that `rf ∪ nsdep` cycles are allowed, where `nsdep ∩ sdep` is the empty set and where `nsdep ∪ sdep = dep`. This means that the compiler is free to replace expressions that are known to always result in a single value with the corresponding constant, without danger of introducing OOTA behavior. We hypothesize that non-speculative code-reordering optimizations are similarly unable to introduce OOTA behavior.

Defining “semantic dependency” sufficiently for formal modeling remains an open issue. In the general case, this the question of whether or not a given dependency is a semantic dependency is of course undecidable. However, this question can be decided straightforwardly in many common cases. One approach would be to flag dependencies that the tool was unable to classify. Another approach would be to consider cases that a given compiler might optimize, and to classify other cases as semantic dependencies.

Asides

The following sections present asides on undecidabilty, inferred ordering, and code generated by old compilers.

Aside on Undecidability

The fact that the choice of the perturbation function is undecidable is no greater obstacle for OOTA than it is for anything else. After all, almost all interesting questions about Turing-complete languages are undecidable. (As Doug Lea pointed out, others are “merely” NP.) The following simple example is a case in point:

```
Thread 1                    Thread 2
--------                    --------
x = 1;                      r1 = y;
if (undecidable())          r2 = x.load(acquire);
    y.store(1, relaxed)
else
    y.store(1, release)
```

Is the outcome `r1 == 1 && r2 ==0` permitted? In general, this is undecidable.

So what do we do about this?

The same things that we have always done. The `ppcmem` and `herd` tools permit only a small finite number of variables, thus avoiding undecidability. The `cbmc` model checker limits the number of passes through each loop, thus considering only finite executions, again avoiding undecidability. These two strategies should also work well for perturbation analysis.

And selection of a perturbation function is usually straightforward:

1. Select the presumed cycle. (Yes, in a large program, there might be a lot of them. Just like there might be a lot of synchronized-with relationships.)
2. Pick a load on the presumed cycle.
3. Select a return value for the load (usually given by the assertion).
4. Check whether the selected return value is consistent, in other words, whether this value results in that same value being stored to that variable. In theory, this step can be undecidable because an overly clever programmer might do something like make the value stored depend on some undecidable proposition such as the halting problem. In practice, making one's program depend on an undecidable proposition seems like a clear case of a deeply flawed design.
5. If the value is consistent, solve for a perturbation function that makes it inconsistent. For most litmus tests, this is at worst simple algebra. Of course, it might be undecidable, in which case it is time to spend some quality time with the litmus test's author. ;-)

So the undecidability should not normally be a problem in practice.

Aside on Inferred Ordering

Suppose that a highly optimizing compiler and a less-aggressive analysis tool are applied to the following litmus test (put forward by Hans Boehm):

```
Thread 1                    Thread 2
--------                    --------
r1 = x;                     r2 = y;
y = r1;                     r3 = f(y);
                            x = r3;
```

Suppose the compiler determined that function `f()` did not represent a semantic dependency, but that the analysis tool was unable to make this determination. Might the analysis tool therefore incorrectly report to the developer that Thread 2's load from `y` is ordered before its store to `x`?

When considering this question, keep in mind that all accesses are relaxed. This means that there are no ordering properties, unless they are supplied by other non-relaxed accesses. Therefore the answer to the question is that the analysis tool should not report that Thread 2's accesses are ordered in any case. It should instead confine itself to disregarding any candidate executions involving OOTA results.

What *can* happen is that the compiler might be able to determine that `f(y)` always returns 42, thus allowing it to transform the code as follows:

```
Thread 1                    Thread 2
--------                    --------
r1 = x;                     r2 = y;
y = r1;                     r3 = 42;
```

```
                        x = r3;
```

The compiler might then generate code that resulted in `x = y = 42`. If the analysis tool had less sophisticated analysis than did the compiler, the tool might well exclude this result. But this would constitute a bug in the tool rather than a problem with OOTA: The compiler correctly determined that the dependency of `r3` on `r2` was not a semantic dependency, while the tool failed to make this distinction. However, a high-quality tool would report that it was unable to prove whether or not `f()` represented a semantic dependency.

Aside on Code From Old Compilers

Suppose that the code for function `f()` in the prior example was generated by an old compiler, perhaps even one that is unaware of C11 and C++11 atomics. Mightn't such a compiler carry out optimizations that could result in OOTA executions? (This possibility was raised in a small-group discussion by Hans Boehm at the 2014 UIUC meeting.)

The answer is "no" for all known compilers used in production. The only possible exception would be research compilers used to investigate value speculation and similar extreme optimizations. However, these research compilers could potentially generate OOTA executions even in sequential code, so it makes sense to exclude them from consideration.

Furthermore, old binaries often introduce data races due to overwriting adjacent fields, which means that old binaries (for example, from gcc 4.6 and earlier) introduce undefined behavior. This means that use of old binaries in concurrent code is a dubious practice in any case.

Nevertheless, this possibility does not permit data-race-free legacy libraries to inflict OOTA executions on multithreaded programs.

Aside on Perverse Compilers

A perverse conforming compiler could recognize a OOTA pattern, and, ignoring the non-normative strictures against OOTA results, provide arbitrary values. However, such a compiler would need an algorithm to recognize the difference between harmful OOTA and benign reordering, and would further need to recognize the difference between consistent and inconsistent OOTA cycles. If such an algorithm existed, it could be used by formal tools to classify cycles. However, this problem is undecidable:

```
Thread 1                Thread 2
--------                --------
r1 = x;                 r2 = y;
y = r1;                 x = r2 + undecidable();
```

Therefore, such a perverse compiler could legally inflict its perversity only in a conservative manner.

Summary

This document has shown that all of the harmful OOTA examples in the the JMM Causality Test Cases are special cases that have a fixed point, and that slight perturbations result in inconsistent results. This supports the hypothesis that any harmful-OOTA test case can be perturbed into an inconsistent state and that benign-reordering test cases cannot be.

This perturbation analysis appears to be equivalent to requiring that `rf ∪ sdep` be acyclic. This is an extremely important result: It means that any compiler optimization that substitutes a constant value for a read known to return that value cannot induce OOTA behavior. This constraint should also ensure that non-speculative code-movemenet optimizations should be similarly unable to induce OOTA behavior.

Effective and efficient modeling of semantic dependencies (`sdep`) remains an important open problem, although there is important work in progress.